

# PRD-07: 用户注册与报名系统

版本: 1.0

日期: 2025-12

状态: 已实施

## 1. 概述

### 1.1 文档目的

本文档描述 Blaze Robotics Academy 用户注册与报名系统的功能需求，包括购物车、报名流程、等待列表、容量管理和过期处理等核心功能。

### 1.2 核心价值

- 便捷性**: 流畅的报名流程，减少用户操作步骤
- 公平性**: 先到先得，防止超售
- 灵活性**: 支持购物车、等待列表等多种报名方式
- 可靠性**: 自动处理过期和容量管理

## 2. 报名状态体系

### 2.1 状态定义

| 状态  | 标识       | 说明         | 过期时间     | 可执行操作        |
|-----|----------|------------|----------|--------------|
| 购物车 | cart     | 在注册清单中，未支付 | 15-30 分钟 | 支付、取消、自动过期   |
| 已保留 | reserved | 已保留（支付中）   | 10-15 分钟 | 完成支付、取消、自动过期 |

|      |            |        |               |                 |
|------|------------|--------|---------------|-----------------|
| 已注册  | enrolled   | 已正式注册  | 无             | 取消（根据取消政策）      |
| 等待列表 | waitlisted | 在等待列表中 | 24-48 小时（通知后） | 升级为 enrolled、取消 |
| 已取消  | cancelled  | 已取消    | 无             | 无               |
| 已过期  | expired    | 已过期    | 无             | 重新加入 cart       |
| 已完成  | completed  | 课程已完成  | 无             | 无               |

2.2 状态流转



2.3 支付状态

| 状态       | 说明  |
|----------|-----|
| unpaid   | 未支付 |
| pending  | 支付中 |
| paid     | 已支付 |
| refunded | 已退款 |

failed

支付失败

## 3. 购物车功能

### 3.1 加入购物车

#### 3.1.1 功能描述

- 入口：课程详情页 → `Enroll Now` 或 `Add to Cart`
- 前提条件：
  - 用户已登录
  - 课程有可用实例
  - 实例未满
  - 满足先修条件（如果要求）
  - 用户未重复报名
- 流程：
  1. 用户选择课程实例
  2. 系统检查容量和先修条件
  3. 创建 `cart` 状态的报名记录
  4. 设置过期时间（15-30 分钟，可配置）
  5. 更新实例的当前学生数（临时占用）
  6. 返回成功消息

#### 3.1.2 用户故事

- 作为 注册用户
- 我希望 将多个课程加入购物车
- 以便 一次性完成支付

### 3.2 购物车页面 ( `/enrollments/cart` )

#### 3.2.1 功能描述

- 显示内容：
  - 课程列表（每个课程显示）：
    - 课程名称

- 实例信息（地点、日期、时间）
  - 价格
  - 剩余时间（倒计时）
  - 删除按钮
- 总价计算
- 操作按钮：
  - Proceed to Checkout：跳转到支付
  - Continue Shopping：返回课程目录
- 实时更新：
  - 显示剩余时间倒计时
  - 过期后自动移除
  - 价格实时计算

### 3.2.2 用户故事

- 作为 注册用户
- 我希望 查看购物车中的所有课程
- 以便 确认报名信息并完成支付

## 3.3 从购物车移除

### 3.3.1 功能描述

- 操作：点击删除按钮
- 流程：
  1. 确认删除
  2. 更新报名状态为 cancelled
  3. 释放实例容量
  4. 刷新购物车列表

---

## 4. 报名流程

### 4.1 结账流程

#### 4.1.1 功能描述

- 入口：购物车页面 → `Proceed to Checkout`
- 流程：
  1. 验证购物车中的所有报名
  2. 检查容量是否仍然可用
  3. 计算总价
  4. 创建 Stripe Checkout Session
  5. 更新报名状态为 `reserved`
  6. 设置保留过期时间（10-15 分钟）
  7. 跳转到 Stripe 支付页面

#### 4.1.2 验证检查

- ☒ 用户已登录
- ☒ 报名记录存在且状态为 `cart`
- ☒ 实例容量仍然可用
- ☒ 先修条件仍然满足
- ☒ 价格计算正确

### 4.2 支付成功处理

#### 4.2.1 Webhook 处理

- 事件： `checkout.session.completed` 、 `payment_intent.succeeded`
- 流程：
  1. 验证 Webhook 签名
  2. 获取报名记录（通过 `enrollment_ids`）
  3. 更新报名状态为 `enrolled`
  4. 更新支付状态为 `paid`
  5. 保存 Stripe 信息（Session ID、Payment Intent ID）
  6. 确认实例容量（正式占用）

#### 4.2.2 成功页面 ( `/enrollments/success` )

- 显示内容：
  - 支付成功消息
  - 已报名的课程列表
  - 课程详情（地点、日期、时间）
  - 操作按钮：
    - `View My Enrollments`：查看所有报名

- Continue Shopping：返回课程目录

## 4.3 支付失败处理

### 4.3.1 Webhook 处理

- 事件：`payment_intent.payment_failed`
- 流程：
  1. 更新支付状态为 `failed`
  2. 保持报名状态为 `reserved`（等待过期或用户重试）
  3. 记录失败原因

### 4.3.2 用户处理

- 选项：
    - 重新尝试支付
    - 取消报名
    - 联系客服
- 

## 5. 等待列表功能

---

### 5.1 加入等待列表

#### 5.1.1 功能描述

- 触发条件：课程实例已满
- 入口：
  - 课程详情页 → `Join Waitlist`
  - 购物车 → 提示加入等待列表
- 流程：
  1. 检查实例是否已满
  2. 创建 `waitlisted` 状态的报名记录
  3. 计算等待列表位置
  4. 设置加入时间
  5. 返回成功消息和位置信息

### 5.1.2 用户故事

- 作为 注册用户
- 我希望 在课程已满时加入等待列表
- 以便 有名额时自动通知我

## 5.2 等待列表通知

### 5.2.1 自动通知

- 触发条件：实例有名额释放
- 流程：
  1. 检查等待列表（按位置排序）
  2. 通知第一个用户（发送邮件）
  3. 设置通知时间
  4. 设置等待列表过期时间（24-48 小时）
  5. 用户需要在过期时间内完成支付

### 5.2.2 通知内容

- 课程名称
- 实例信息（地点、日期、时间）
- 等待列表位置
- 支付链接
- 过期时间

## 5.3 等待列表升级

### 5.3.1 自动升级

- 触发条件：用户完成支付
  - 流程：
    1. 更新报名状态为 `enrolled`
    2. 更新支付状态为 `paid`
    3. 确认实例容量
    4. 通知下一个等待列表用户（如果有）
-

## 6. 容量管理

---

### 6.1 容量计算

#### 6.1.1 可用容量

```
可用容量 = max_students - current_students - reserved_count - cart_count
```

#### 6.1.2 容量检查

- 加入购物车时：检查可用容量 > 0
- 结账时：再次检查可用容量
- 支付成功时：正式占用容量

### 6.2 容量更新

#### 6.2.1 自动更新

- 触发器：报名状态变更时自动更新
- 更新规则：
  - `cart` → `enrolled`：占用容量
  - `reserved` → `enrolled`：占用容量
  - `waitlisted` → `enrolled`：占用容量
  - `cart` → `cancelled`：释放容量
  - `reserved` → `cancelled`：释放容量

### 6.3 并发控制

#### 6.3.1 防止超售

- 数据库约束：唯一约束防止重复报名
- 乐观锁：检查容量时使用版本号
- 事务处理：确保容量更新的原子性

---

## 7. 过期处理

---



## 7.1 购物车过期

### 7.1.1 过期时间

- 默认：15-30 分钟（可配置）
- 配置位置：`enrollment_config` 表

### 7.1.2 过期处理

- 后台任务：Cron Job 定期处理
- 流程：
  1. 查找过期的 `cart` 状态报名
  2. 更新状态为 `expired`
  3. 释放实例容量
  4. 记录过期时间

## 7.2 保留过期

### 7.2.1 过期时间

- 默认：10-15 分钟（可配置）

### 7.2.2 过期处理

- 流程：
  1. 查找过期的 `reserved` 状态报名
  2. 更新状态为 `expired`
  3. 更新支付状态为 `failed`
  4. 释放实例容量

## 7.3 等待列表过期

### 7.3.1 过期时间

- 默认：24-48 小时（通知后）

### 7.3.2 过期处理

- 流程：

1. 查找过期的 `waitlisted` 状态报名
  2. 更新状态为 `expired`
  3. 通知下一个等待列表用户
- 

## 8. 先修条件检查

---

### 8.1 检查时机

#### 8.1.1 加入购物车时

- 检查：用户是否完成所有必需的先修课程
- 如果不满足：
  - 返回错误消息
  - 显示缺失的先修课程列表
  - 提供先修课程链接

#### 8.1.2 结账时

- 再次检查：确保先修条件仍然满足
- 如果不满足：阻止结账

### 8.2 先修条件类型

#### 8.2.1 Required (必需)

- 规则：必须完成才能报名
- 检查：用户必须有这些课程的 `completed` 状态报名

#### 8.2.2 Recommended (推荐)

- 规则：建议完成，但不强制
  - 检查：不阻止报名，但显示提示
- 

## 9. 用户故事

---

## 9.1 用户故事

### 故事 1：浏览并报名课程

- 作为 注册用户
- 我希望 浏览课程并加入购物车
- 以便 一次性完成多个课程的支付

### 故事 2：查看购物车

- 作为 注册用户
- 我希望 查看购物车中的所有课程
- 以便 确认报名信息

### 故事 3：完成支付

- 作为 注册用户
- 我希望 安全地完成支付
- 以便 正式注册课程

### 故事 4：加入等待列表

- 作为 注册用户
- 我希望 在课程已满时加入等待列表
- 以便 有名额时自动通知我

## 9.2 管理员故事

### 故事 1：管理报名

- 作为 管理员
- 我希望 查看和管理所有报名
- 以便 跟踪报名状态和处理问题

### 故事 2：处理过期报名

- 作为 系统
  - 我希望 自动处理过期的报名
  - 以便 释放容量并通知等待列表用户
-

## 10. 非功能需求

---

### 10.1 性能要求

- 加入购物车：< 1 秒
- 购物车加载：< 1 秒
- 结账处理：< 2 秒

### 10.2 可靠性要求

- 防止超售：确保不会超过容量
- 数据一致性：确保容量和报名状态一致
- 错误处理：友好的错误消息

### 10.3 安全性要求

- 用户验证：确保用户只能操作自己的报名
  - 容量验证：防止并发超售
  - 支付安全：使用 Stripe 安全支付
- 

## 11. 技术实现

---

### 11.1 数据模型

#### 11.1.1 核心表

- `course_enrollments`：报名表
- `enrollment_status_history`：状态历史表
- `waitlist_notifications`：等待列表通知表
- `enrollment_config`：配置表

#### 11.1.2 关键字段

- `status`：报名状态
- `payment_status`：支付状态
- `cart_expires_at`：购物车过期时间

- `reserved_expires_at` : 保留过期时间
- `waitlist_position` : 等待列表位置
- `waitlist_expires_at` : 等待列表过期时间

## 11.2 API 端点

### 11.2.1 用户 API

- `GET /api/enrollments/cart` : 获取购物车
- `POST /api/enrollments/cart` : 加入购物车
- `DELETE /api/enrollments/cart/[id]` : 从购物车移除
- `POST /api/enrollments/waitlist` : 加入等待列表
- `DELETE /api/enrollments/waitlist/[id]` : 从等待列表移除
- `GET /api/user/enrollments` : 获取所有报名
- `GET /api/user/courses/[id]/can-enroll` : 检查是否可以报名

### 11.2.2 支付 API

- `POST /api/payments/create-checkout-session` : 创建支付会话
- `GET /api/payments/success` : 支付成功验证
- `POST /api/payments/webhook` : Stripe Webhook

### 11.2.3 后台任务 API

- `POST /api/cron/process-expired-enrollments` : 处理过期报名

## 11.3 后台任务

### 11.3.1 过期处理任务

- 频率: 每 5 分钟 (可配置)
  - 功能:
    - 处理过期的购物车
    - 处理过期的保留
    - 处理过期的等待列表
    - 通知等待列表用户
-

## 12. 验收标准

---

### 12.1 功能验收

-  可以加入购物车
-  可以查看购物车
-  可以完成支付
-  可以加入等待列表
-  过期自动处理
-  容量管理正确
-  先修条件检查正确

### 12.2 数据完整性

-  防止超售
  -  状态转换正确
  -  容量更新正确
- 

## 13. 未来扩展

---

### 13.1 计划功能

-  报名提醒（课程开始前）
  -  自动重新报名（如果课程取消）
  -  报名转赠功能
  -  批量报名功能
- 

## 14. 附录

---

### 14.1 相关文档

- `COURSE_ENROLLMENT_DESIGN.md`：报名系统设计
- `ENROLL_BUTTON_DISABLED_CONDITIONS.md`：报名按钮禁用条件

- `ENROLLMENT_IMPLEMENTATION_STATUS.md` : 实施状态

## 14.2 数据库脚本

- `create-enrollment-tables.sql` : 报名表创建脚本